Introduction to C

What is C Language?

C is a general-purpose computer programming language.

C is also said to be structured programming language or function oriented programming language.

C is a High level programming language.

Why do we use C?

C language is used to create applications or software.

Initially, C was developed to create an operating system called UNIX.

The popular software like **Linux OS**, **PHP** & **MySQL** are created using C language.

Why do we use C?

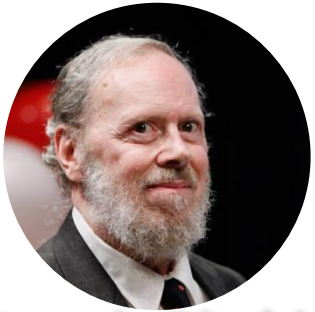
Generally C Language is used to create the following…

- Operating Systems
- Language Compilers
- Assemblers
- Interpreters
- Text Editors
- Network Drivers
- Databases

History of C Language?

C Language was developed by Dennis Ritchie in the year of 1972.

Born – September 9, 1941
Profession – Computer Scientist
Place – Bell Labs, US
Known As – Father of C & UNIX
Awards – **Turing Award(1983)**
National Medal of Technology(1998)
IEEE Medal(1990)
Computer Pioneer Award(1994)
Computer History Museum Fellow(1997)
Harold Pender Award(2003)
Died – October 12, 2011



Dennis Ritchie

History of C Language?

Let's see the programming languages that were developed before C language…

CPL – Common Programming Language invented by Martin Richards in 1960's
BCPL – Basic Combined Programming Language by Martin Richards in 1966
B Language – by Ken Thompson & Dennis Ritchie in 1969
Traditional C – by Dennis Ritchie in 1972
K&R C – by Kernighan & Dennis Ritchie in 1978
ANSI C – by ANSI Committee in 1989
ANSI/ISO C – by ISO Committee in 1990
C99 – by Standardization Committee in 1999



Ken Thompson



Dennis Ritchie

Where C stands

Programming languages are divided into two

- Problem oriented languages or High-level languages

Objective – better programming efficiency or **faster program development**

Eg. FORTRAN, BASIC, PASCAL etc….

- Machine oriented languages or Low level languages

Objective – better machine efficiency or **faster program execution**

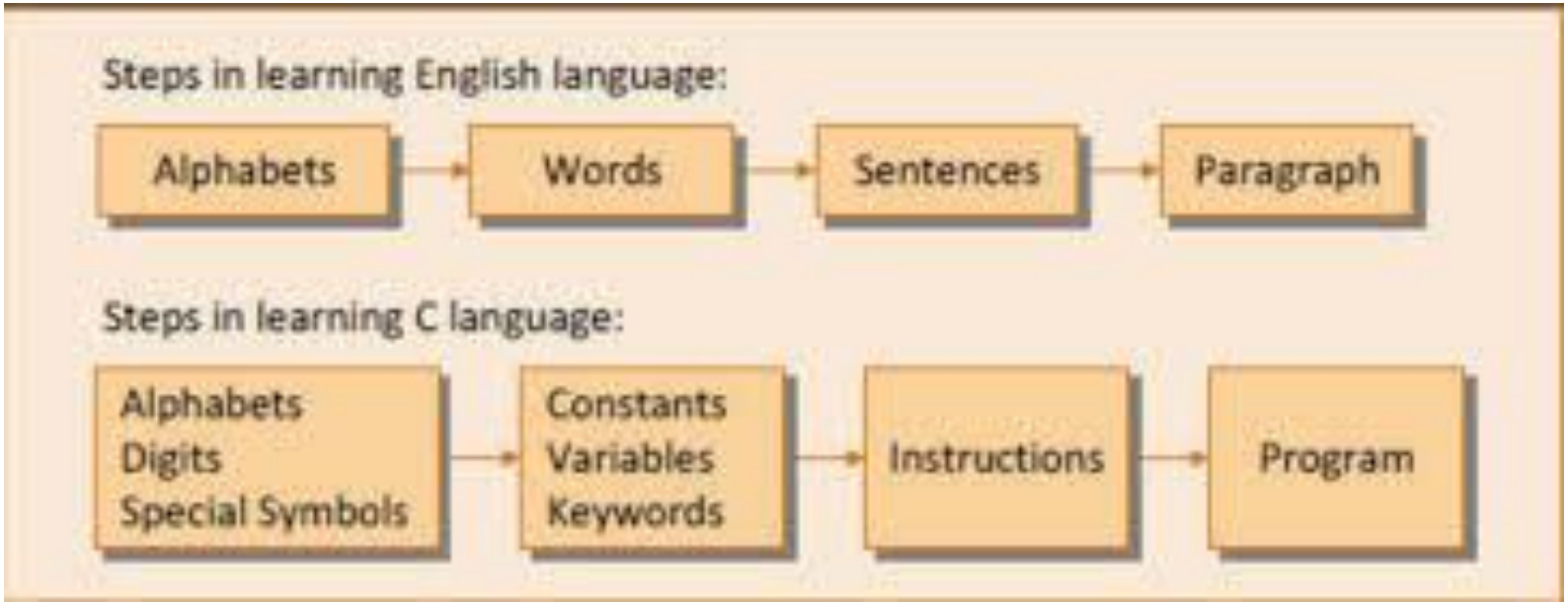
Eg. Machine language and Assembly language

C stands in between these two categories. Hence called **middle level language. Designed for both – good programming efficiency** and **good machine efficiency**

Characteristics

- C is a High level , general –purpose structured programming language. Instructions of C consists of terms that are very closely same to algebraic expressions, consisting of certain English keywords such as if, else, for ,do and while
- C contains certain additional features that allows it to be used at a lower level , acting as bridge between machine language and the high level languages.
- This allows C to be used for system programming as well as for applications programming

Steps in learning C



“C” language TOKENS

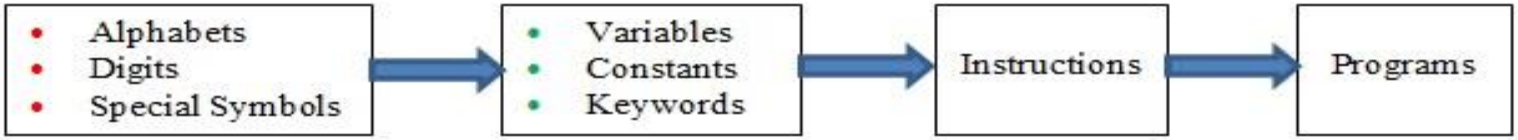
- ✓ The smallest individual units in a C program are known as tokens. In a C source program, the basic element recognized by the compiler is the "token." A token is source-program text that the compiler does not break down into component elements.
- ✓ C has 6 different types of tokens viz.

1. Keywords [e.g. float, int, while]
2. Identifiers [e.g. main, amount]
3. Constants [e.g. -25.6, 100]
4. Strings [e.g. “SMIT”, “year”]
5. Special Symbols [e.g. {, }, [,]]
6. Operators [e.g. +, -, *]



- ✓ C - programs are written using these tokens and the general syntax.

STEPS IN LEARNING C



One must understand character set used while writing C programs. Characters in C are grouped into the following categories:

1. **Letters or Alphabets**
   A, B, C,.....Z
   a, b, c, ...z

2. **Digits**
   0, 1, 2, 3, 4, 5, 6, 7, 8, 9

3. **Special symbols or characters**
   + - * # $ @ ? . , { ([})] etc

4. **White spaces**: also known as **blanks** or **blank spaces**

Identifiers

A 'C' program consist of two types of elements, user defined and system defined. Identifiers is nothing but a name given to these elements.

- An **identifier is a word used by a programmer to name a variable, function, or label.**
- Identifiers consist of letters and digits if any order, except that the first character must be letter.
- Both Upper and lowercase letters can be used

Keywords

Keywords are words that have special meaning to the C compiler. Keywords are reserved words of the language.

- Their meaning cannot be changed at any instance
- Serve as basic building block for program statements
- All keywords are written in **lowercase**
- 32 Keywords in C Programming

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Variables

A variable is nothing but a name given to a storage area that our programs can manipulate.

- Each variable in C has a specific type, which determines the size and layout of the variable's memory; the range of values that can be stored within that memory; and the set of operations that can be applied to the variable.
- The name of a variable can be composed of letters, digits, and the underscore character.
- Variables are identified by only first 32 characters.
- It must begin with either a letter or an underscore.
- Must not be a keyword
- Upper and lowercase letters are distinct because C is case-sensitive.
- No commas or blanks are allowed within a variable name
- No special symbol other than underscore can be used as a variable name.
- Eg. int num1, char code, int st_rank etc…

Variables

- There are following basic variable types −

Type	Description
char	Typically a single octet(one byte). This is an integer type.
int	The most natural size of integer for the machine.
float	A single-precision floating point value.
double	A double-precision floating point value.
void	Represents the absence of type.

Good practice – naming variables by using meaningful variable names

Data types in 'ansi c'

- There are three classes of data types here::

- Primitive data types
  - int, float, double, char
- Aggregate OR derived data types
  - Arrays come under this category
  - Arrays can contain collection of int or float or char or double data
- User defined data types
  - Structures and enum fall under this category.

Data Types- different attributes

Type	Size	Representation	Minimum range	Maximum range
char, signed char	8 bits	ASCII	-128	127
unsigned char	bool 8 bits	ASCII	0	255
short, signed short	16 bits	2's complement	-32768	32767
unsigned short	16 bits	Binary	0	65535
int, signed int	16 bits	2's complement	-32768	32767
unsigned int	16 bits	Binary	0	65535
long, signed long	32 bits	2's complement	-2,147,483,648	2,147,483,647
unsigned long	32 bits	Binary	0	4,294,967,295
float	32 bits	IEEE 32-bit	1.175495e-38	3.4028235e+38
double	32 bits	IEEE 32-bit	1.175495e-38	3.4028235e+38
long double	32 bits	IEEE 32-bit	1.175495e-38	3.4028235e+38

Basic Data Types

- Integral Types
  - Integers are stored in various sizes. They can be signed or unsigned.
  - **Example**
    Suppose an integer is represented by a byte (8 bits). Leftmost bit is sign bit. If the sign bit is 0, the number is treated as positive.
    Bit pattern 01001011 = 75 (decimal).
    The largest positive number is 01111111 = $2^7 - 1 = 127$.
    Negative numbers are stored as two's complement or as one's complement.
    -75 = 10110100 (one's complement).
    -75 = 10110101 (two's complement).

Basic Data Types

- Integral Types
  - **char** Stored as 8 bits. Unsigned 0 to 255. Signed -128 to 127.
  - **short int** Stored as 16 bits. Unsigned 0 to 65535. Signed -32768 to 32767.
  - **int** Same as either short or long int.
  - **long int** Stored as 32 bits. Unsigned 0 to 4294967295. Signed -2147483648 to 2147483647

Basic Data Types

- Floating Point Numbers
  - Floating point numbers are rational numbers. Always signed numbers.
  - **float** Approximate precision of 6 decimal digits .
    - Typically stored in 4 bytes with 24 bits of signed mantissa and 8 bits of signed exponent.
  - **double** Approximate precision of 14 decimal digits.
    - Typically stored in 8 bytes with 56 bits of signed mantissa and 8 bits of signed exponent.

Constants

A constant is a value or an identifier whose value cannot be altered in a program. For example: 1, 2.5,
As mentioned, an identifier also can be defined as a constant. eg. const double PI = 3.14
Here, PI is a constant. Basically what it means is that, PI and 3.14 is same for this program.

Integer constants

A integer constant is a numeric constant (associated with number) without any fractional or exponential part. There are three types of integer constants in C programming:

decimal constant(base 10)
octal constant(base 8)
hexadecimal constant(base 16)

Rules

1) *Must have at least one digit*
2) *Must not have a decimal point*
3) *Could be either positive or negative*
4) *If no sign precedes, assumed to be positive*
5) *No commas or blanks are allowed*
6) *Allowable range -32768 to +32768*

Constants

Floating-point constants

A floating point constant is a numeric constant that has either a fractional form or an exponent form.
For example: 2.0,0.0000234,-0.22E-5, -32.76……

Rules

1) *Must have at least one digit*
2) *Must have a decimal point*
3) *Could be either positive or negative*
4) *If no sign precedes, assumed to be positive*
5) *No commas or blanks are allowed*

Character constants

A character constant is a constant which uses single quotation around characters. For example: 'a', 'l', 'm', 'F‘, ‘=‘, ‘7’……

Rules

1) *Must be a single alphabet or a single digit or a special symbol enclosed within single inverted comma.*
2) *Max. length is 1 character*

String constants

String constants are the constants which are enclosed in a pair of double-quote marks.
For example: "good" ,"x", "Earth is round”

Escape Sequences

Sometimes, it is necessary to use characters which cannot be typed or has special meaning in C programming. For example: newline(enter), tab, question mark etc. In order to use these characters, escape sequence is used.

For example: \n is used for newline. The backslash (\ ) causes "escape" from the normal way the characters are interpreted by the compiler.

Escape Sequences	Character
\b	Backspace
\f	Form feed
\n	Newline
\r	Return
\t	Horizontal tab
\v	Vertical tab
\\	Backslash
\'	Single quotation mark
\"	Double quotation mark
\?	Question mark
\0	Null character

Operators

An operator is a symbol which operates on a value or a variable.
For example: + is an operator to perform addition.

Operators can be classified as:

- Arithmetic Operators
- Increment and Decrement Operators
- Assignment Operators
- Relational Operators
- Logical Operators
- Conditional Operators
- Bitwise Operators
- Special Operators

C Instructions

Four types of instructions:

a) Type declaration instruction
b) Input/output instruction
c) Arithmetic instruction
d) Control instruction

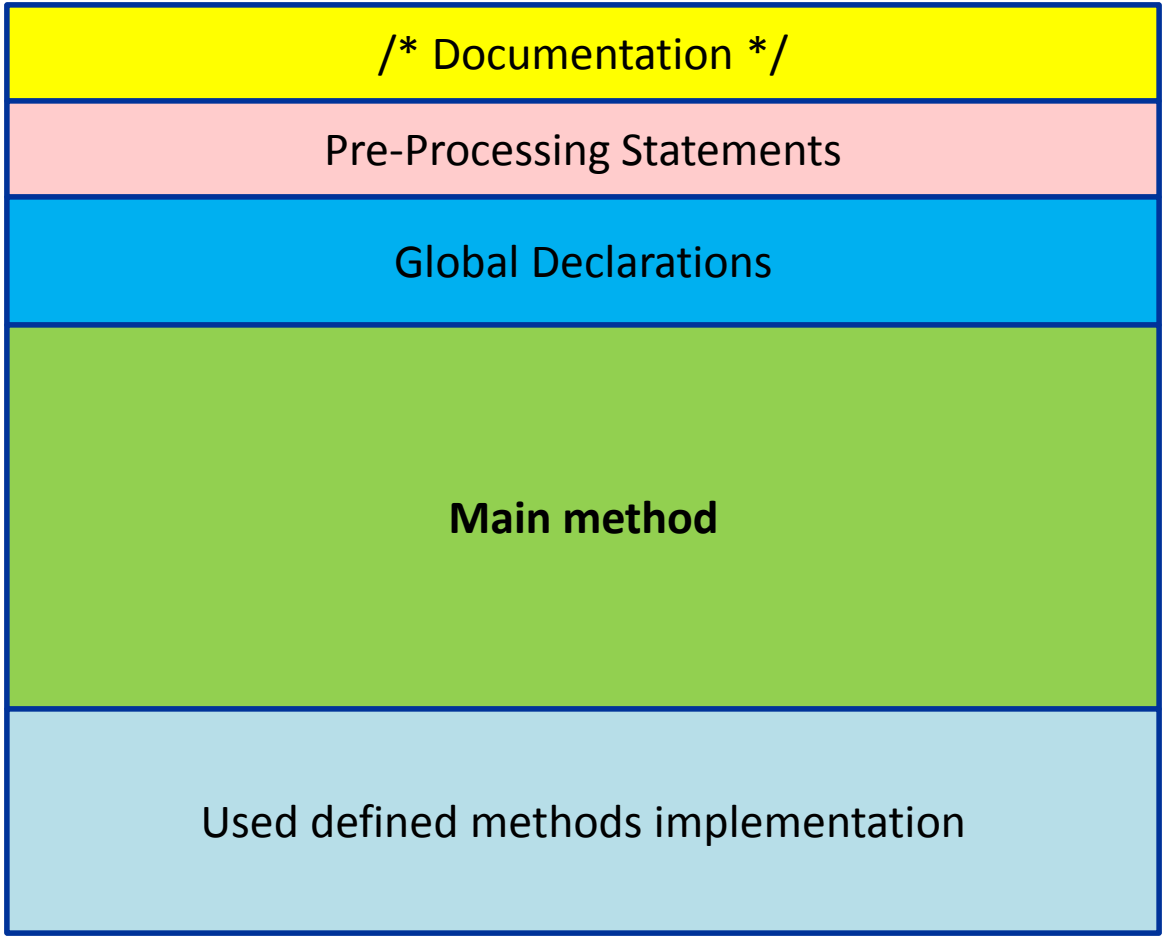
Type declaration instruction
To declare the type of variables used in a C program

Input/output instruction
To perform the function of supplying input data to a program and obtaining the output results from it

Arithmetic instruction
To perform Arithmetic operations between constants and variables

Control instruction
To control the sequence of execution of various statements in a C program.

General Structure of a C Program



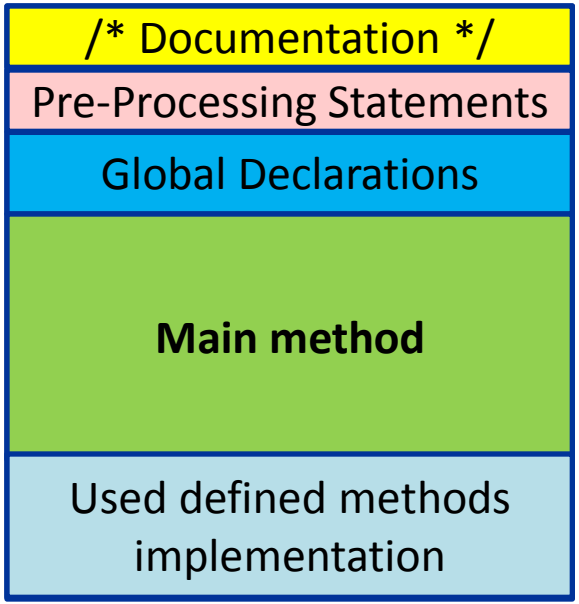
General Structure of a C Program

Documentation

- It is used to provide brief information of the program.

- This part is written using **comments**.

- Generally the documentation part does not executed by compiler & it is optional part.



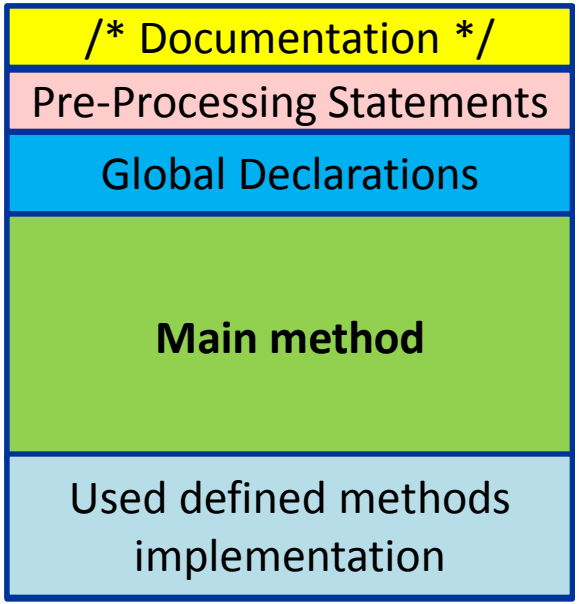
General Structure of a C Program

Pre-Processing Statements

- It is used to link the header files, define the constants, etc...

- Every preprocessor statement starts with hash (#) symbol.

- Every Preprocessor statement tells to the compiler to perform required pre-processing before the actual compilation.

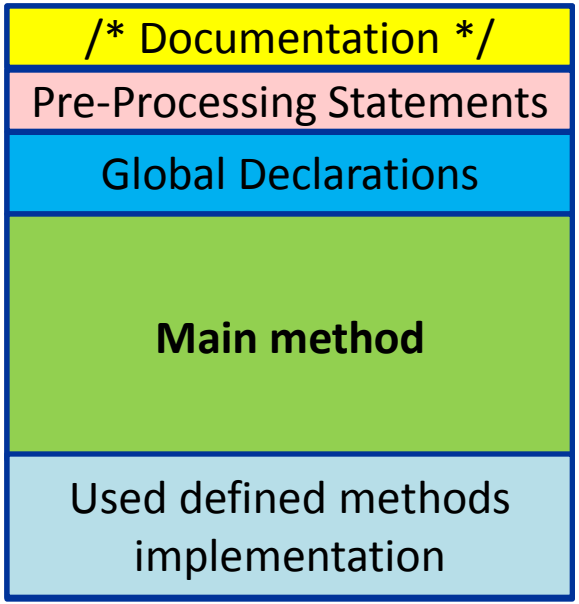


General Structure of a C Program

Pre-Processing Statements

- Examples

  #include
  #define
  #if
  #else



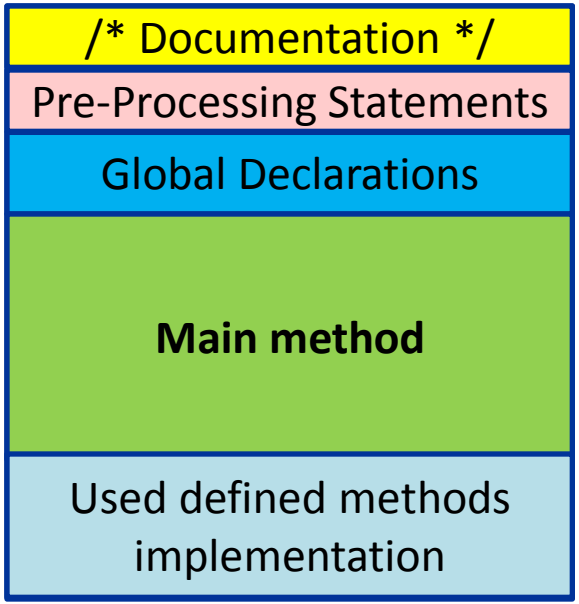
General Structure of a C Program

Global Declaration

- This part is used to declare the variables which are common for multiple methods.

- In this section, we also declare enumeration, structure, unions, userdefined methods etc...

- It is also optional part. According to our requirement we write this section.



General Structure of a C Program

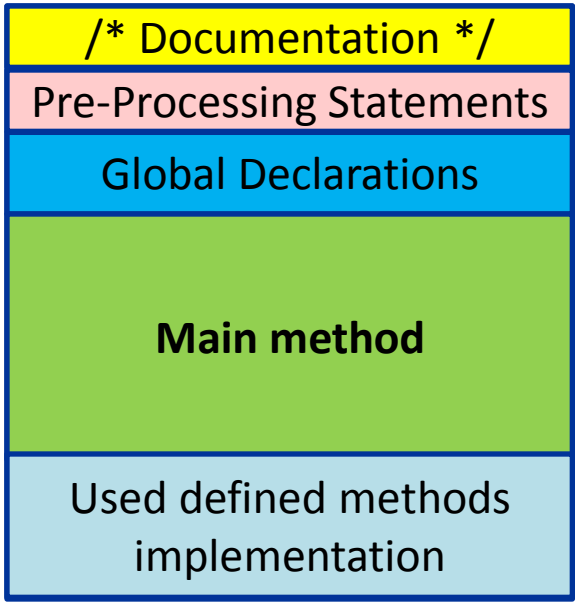
Main method

- main method is the compulsory part for any c program.

- C language is a function oriented programming language, so every c program must have at least one function and that must be main.

- Main is a user-defined method which specifies the starting point of the program execution.

- Every c program execution starts with main method and ends with main method itself.



General Structure of a C Program

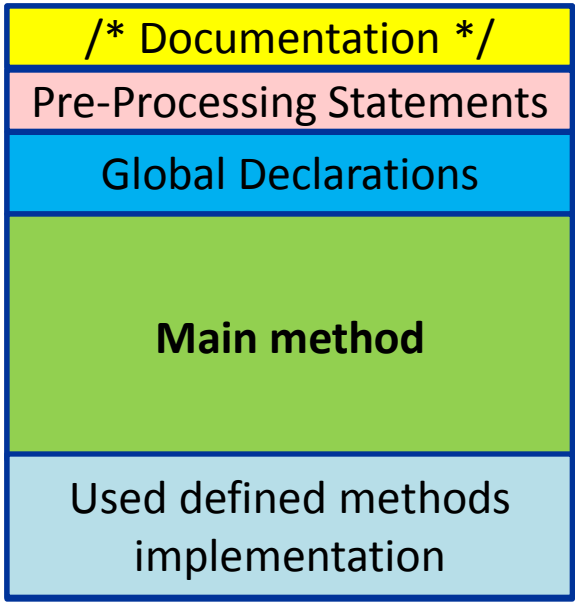
User-defined Methods - Functions

- In this section of the program we write the actual code for the user-defined methods.

- User-defined methods can be implemented either before or after the method.

- If it is implemented after the main then it must be declared either inside or before the main method.

- If it is implemented before the main then the declaration can be ignored.



General Structure of a C Program - Example

```
/* Program to print a message Hello World! */

#include<stdio.h>

void main()
{

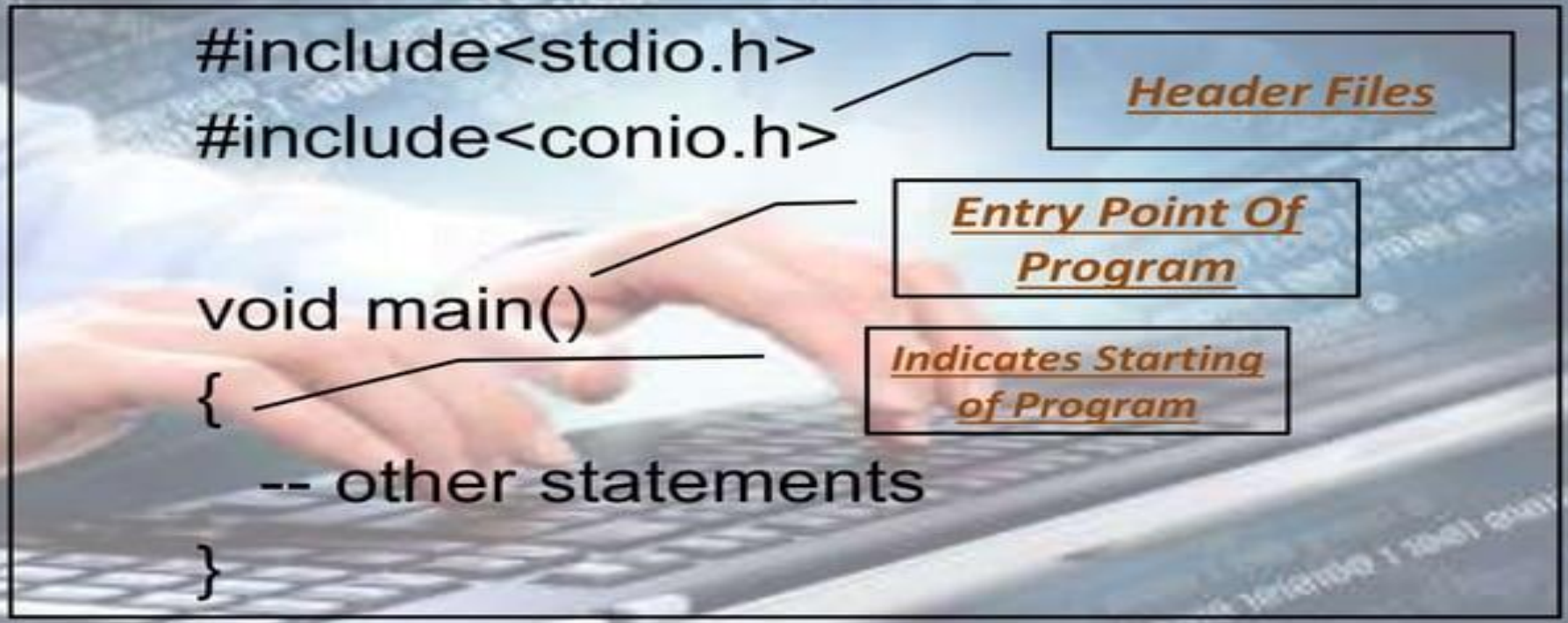
      printf("Hello World!!!!");

}
```

Rules for Writing C programs

- Every C program must contain exact one main method
- Every executable instruction must end with semicolon (;)
- All the system defined words (Keywords) must be used in lowercase letters
- Every open brace ({) must have the respective closing brace (})
- The variables must be declared in the declaration section before they are used

Basic Structure Of "C" Programs



Header files

- ✓ The files that are specified in the include section is called as Header File.
- ✓ These are precompiled files that has some functions defined in them.
- ✓ We can call those functions in our program by supplying parameters.
- ✓ Header file is given an extension .h .
- ✓ C Source file is given an extension .c .

Main function

- ✓ This is the *"Entry Point"* of a program.
- ✓ When a file is executed, the start point is the main function.
- ✓ From main function the flow goes as per the programmers choice.
- ✓ There may or may not be other functions written by user in a program.
- ✓ Main function is compulsory for any C program.

Declaring Variables

- ✓ Before using a variable, you must give the compiler some information about the variable; i.e., you must declare it.
- ✓ The declaration statement includes the data type of the variable.
- ✓ Examples of variable declarations:

```
int  length ;
float  area ;
```

- ✓ Variables are not automatically initialized. For example, after declaration

```
int sum;
```

the value of the variable sum can be anything (garbage).

- ✓ Thus, it is good practice to initialize variables when they are declared.
- ✓ Once a value has been placed in a variable it stays there until the program alters it.

Declaring a Variable

Each variable used must be declared.

A form of a declaration statement is

data-type var1, var2,…;

Declaration announces the data type of a variable and allocates appropriate memory location. No initial value (like 0 for integers) should be assumed.

It is possible to assign an initial value to a variable in the declaration itself.

data-type var = expression;

Examples

int sum = 0;

char newLine = ‘\n’;

Global and Local Variables

- Global Variables
  - These variables are declared outside all functions.
  - Life time of a global variable is the entire execution period of the program.
  - Can be accessed by any function defined below the declaration, in a file.

```
#include <stdio.h>
float pi = 3.14;    /* Global variable */

main()
{
  float rad; /* Local variable*/
  …….
}
```

- Local Variables
  - These variables are declared inside some functions.
  - Life time of a local variable is the entire execution period of the function in which it is defined.
  - Cannot be accessed by any other function.
  - In general variables declared inside a block are accessible only in that block.

Running a 'C' Program

- Type a program.
- Save it.
- Compile the program – This will generate an .exe file (executable)
- Run the program (Actually the exe created out of compilation will run and not the .c file)
- In different compiler we have different option for compiling and running.

C Program Phases

- Editor - code by programmer
- Compiling using gcc:
  - Preprocess – expand the programmer's code
  - Compiler – create machine code for each file
  - Linker – links with libraries and all compiled objects to make executable
- Running the executable:
  - Loader – puts the program in memory to run it
  - CPU – runs the program instructions

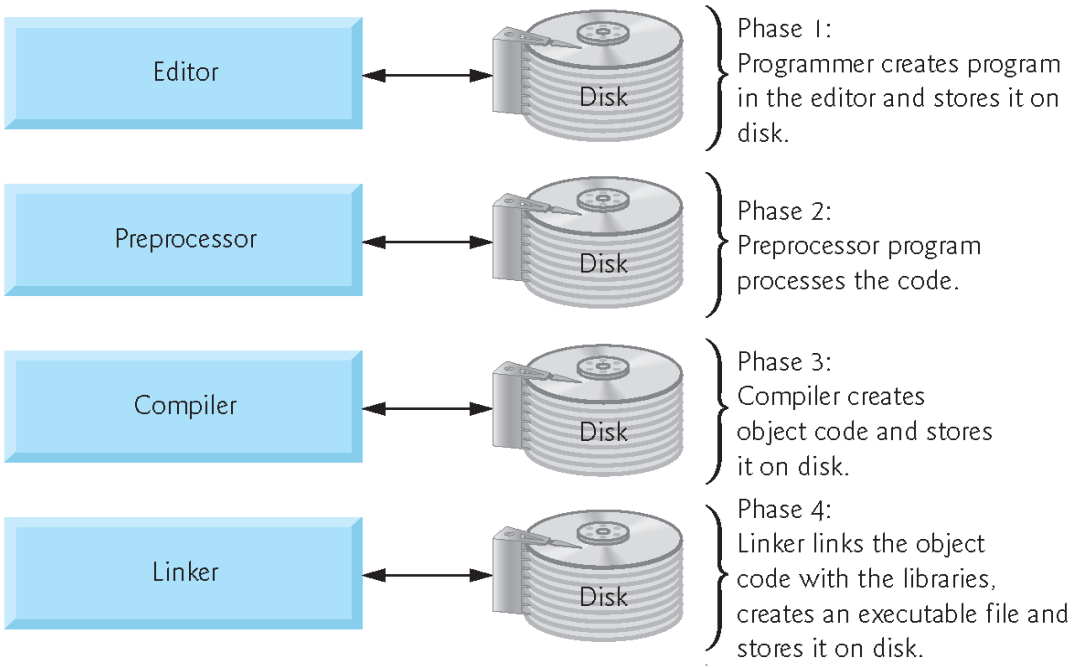


Fig. 1.7 | Typical C development environment. (Part 1 of 3.)

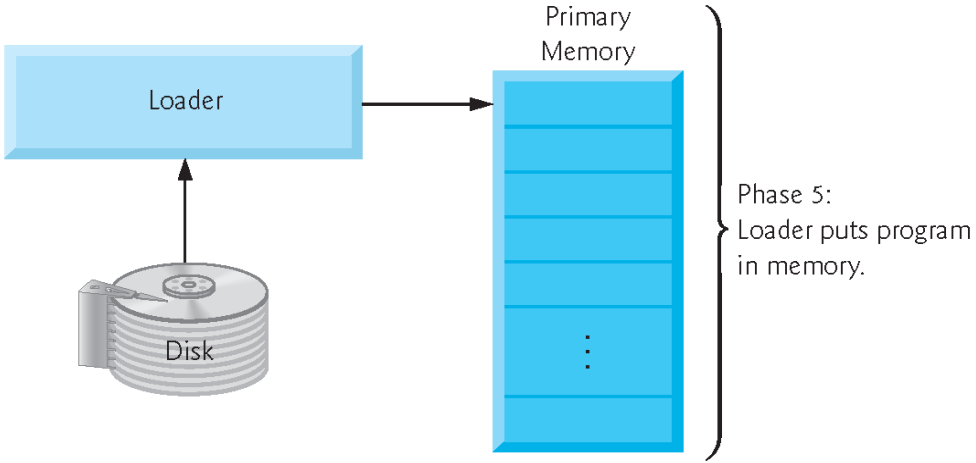


Fig. 1.7 | Typical C development environment. (Part 2 of 3.)

Operators

- **Arithmetic Operators**
  - **+, - , *,** / and the modulus operator %.
  - + and – have the same precedence and associate left to right.
    3 – 5 + 7 = (3 – 5) + 7 ≠ 3 – (5 + 7)
    3 + 7 – 5 + 2 = ((3 + 7) – 5) + 2
  - ***, /, %** have the same precedence and associate left to right.
  - The **+, -** group has lower precedence than the ***, / %** group.
    3 – 5 * 7 / 8 + 6 / 2
    3 – 35 / 8 + 6 / 2
    3 – 4.375 + 6 / 2
    3 – 4.375 + 3
    -1.375 + 3
    1.625

Operators

- **Arithmetic Operators**
  - % is a modulus operator. x % y results in the remainder when x is divided by y and is zero when x is divisible by y.
  - **Cannot be applied to float or double variables.**
  - Example

```
if ( num % 2 == 0 )
   printf(“%d is an even number\n”, num)’;
else
   printf(“%d is an odd number\n”, num);
```

Operators

- **Relational Operators**
  - <, <=, > >=, ==, != are the relational operators. The expression

    operand1 relational-operator operand2

    takes a value of 1(int) if the relationship is true and 0(int) if relationship is false.
  - Example

    int a = 25, b = 30, c, d;

    c = a < b;

    d = a > b;

    value of c will be 1 and that of d will be 0.

Operators

- **Logical Operators**
  - &&, || and ! are the three logical operators.
  - expr1 && expr2 has a value 1 if expr1 and expr2 both are nonzero.
  - expr1 || expr2 has a value 1 if expr1 and expr2 both are nonzero.
  - !expr1 has a value 1 if expr1 is zero else 0.
  - Example
  - if (marks >= 40 && attendance >= 75) grade = ‘P’
  - If (marks < 40 || attendance < 75) grade = ‘N’

Operators

- **Assignment operators**
  - The general form of an assignment operator is
  - v op= exp
  - Where v is a variable and op is a binary arithmetic operator. This statement is equivalent to
  - v = v op (exp)
  - a = a + b can be written as a += b
  - a = a * b can be written as a *= b
  - a = a / b can be written as a /= b
  - a = a - b can be written as a -= b

Operators

- **Increment and Decrement Operators**
  - The operators ++ and –- are called increment and decrement operators.
  - a++ and ++a are equivalent to a += 1.
  - a-- and --a are equivalent to a -= 1.
  - ++a op b is equivalent to a ++; a op b;
  - a++ op b is equivalent to a op b; a++;
  - Example

    Let b = 10 then

    (++b)+b+b = 33

    b+(++b)+b = 33

    b+b+(++b) = 31

    b+b*(++b) = 132

Type Conversions

- The operands of a binary operator must have a the same type and the result is also of the same type.
- Integer division:

  c = (9 / 5)*(f - 32)

  The operands of the division are both int and hence the result also would be int. For correct results, one may write

  c = (9.0 / 5.0)*(f - 32)
- In case the two operands of a binary operator are different, but compatible, then they are converted to the same type by the compiler. The mechanism (set of rules) is called Automatic Type Casting.

  c = (9.0 / 5)*(f - 32)
- It is possible to force a conversion of an operand. This is called Explicit Type casting.

  c = ((float) 9 / 5)*(f - 32)

Automatic Type Casting

1. char and short operands are converted to int
2. Lower data types are converted to the higher data types and result is of higher type.
3. The conversions between unsigned and signed types may not yield intuitive results.

Example

```
float f; double d; long l;
int i; short s;
d + f       f will be converted to double
i / s       s will be converted to int
l / i       i is converted to long; long result
```

Hierarchy

double

float

long

int

short and char

Explicit Type Casting

- The general form of a type casting operator is
- **(type-name) expression**
- It is generally a good practice to use explicit casts than to rely on automatic type conversions.
- Example

  C = (float)9 / 5 * (f – 32)
- float to int conversion causes truncation of fractional part
- double to float conversion causes rounding of digits
- long int to int causes dropping of the higher order bits.

Precedence and Order of evaluation

Table 3.8 Summary of C Operators

OPERATOR	DESCRIPTION	ASSOCIATIVITY	RANK
( ) [ ]	Function call Array element reference	Left to right	1
+ − ++ −− ! ~ * & sizeof (type)	Unary plus Unary minus Increment Decrement Logical negation Ones complement Pointer reference (indirection) Address Size of an object Type cast (conversion)	Right to left	2
* / %	Multiplication Division Modulus	Left to right	3
+ −	Addition Subtraction	Left to right	4

Precedence and Order of evaluation

OPERATOR	DESCRIPTION	ASSOCIATIVITY
<< >>	Left shift Right shift	Left to right
< <= > >=	Less than Less than or equal to Greater than Greater than or equal to	Left to right
== !=	Equality Inequality	Left to right
&	Bitwise AND	Left to right
^	Bitwise XOR	Left to right
|	Bitwise OR	Left to right
&&	Logical AND	Left to right
||	Logical OR	Left to right
?:	Conditional expression	Right to left
= *= /= %= += -= &= ^= |= <<= >>=	Assignment operators	Right to left
,	Comma operator	Left to right